

Prueba – Analista de QA

Ejercicio práctico – Digital Onboarding

Para este ejercicio planteo una estrategia basada principalmente en mi experiencia en QA funcional, pruebas de servicios y validación de requerimientos. La automatización la considero como un apoyo para los casos que son repetitivos y estables, y la incorporaría de forma gradual.

1. Enfoque general de pruebas

Primero identificaría los procesos críticos del negocio y los riesgos asociados. Después definiría qué validar manualmente, qué escenarios conviene automatizar y qué evidencias necesito para decidir si una versión puede avanzar.

Prioridades principales:

- Flujo completo de onboarding y creación del comercio.
- Pagos, confirmaciones y prevención de cobros duplicados.
- Inventario y consistencia de los estados.
- Integraciones entre API, colas y servicios externos.
- Manejo de errores, reintentos y estados intermedios.
- Seguridad y exposición de información sensible.
- Rendimiento de los servicios críticos.

2. Niveles de prueba

Distribuiría las pruebas aproximadamente de la siguiente manera, entendiendo que la proporción puede cambiar según el producto y sus riesgos:

- **Pruebas funcionales/manuales: 50%.** Validación de requerimientos, reglas de negocio, errores y escenarios nuevos.
- **Pruebas API e integración: 25%.** Validación de servicios, respuestas, errores e integraciones.
- **Pruebas E2E: 15%.** Flujos principales de usuario de punta a punta.
- **Performance y otras pruebas: 10%.** Validaciones de rendimiento y escenarios específicos según el riesgo.

Durante los primeros 6 meses no automatizaría todo. Empezaría con escenarios estables, repetitivos y de mayor impacto. Dejaría para después los casos que cambian constantemente o que tienen poco riesgo. Primero buscaría que las pruebas automatizadas sean confiables y fáciles de mantener.

3. Datos y ambientes

Usaría datos sintéticos y controlados para las pruebas. Mantendría separados los ambientes y evitaría utilizar credenciales o información real en el repositorio.

Para escenarios repetibles, prepararía datos de prueba conocidos y, cuando sea posible, permitiría que cada ejecución cree y elimine sus propios datos. Esto facilita la repetición de las pruebas y evita depender de datos que otra persona pueda modificar.

4. Procesamiento asíncrono

Como existen colas y procesos que pueden ejecutarse de manera asíncrona, no basaría las pruebas en esperas fijas como “esperar 10 segundos”. Preferiría consultar el estado o esperar una condición concreta, con un tiempo máximo definido.

- Validar el procesamiento correcto del mensaje.
- Validar reintentos cuando exista un fallo temporal.
- Validar mensajes duplicados.

- Validar estados intermedios y recuperación.
- Mantener trazabilidad mediante un tracking ID sin exponer información sensible.

5. Integraciones externas

Para terceros como servicios de pago, identidad o firma digital, utilizaría mocks, stubs, simuladores o contratos cuando sea necesario. Así puedo validar diferentes respuestas sin depender de la disponibilidad del proveedor real.

6. Estrategia de automatización

La automatización se implementaría de manera progresiva. Empezaría por API y procesos críticos porque permiten validaciones repetibles y rápidas. Después incorporaría algunos flujos E2E que representen recorridos importantes del usuario.

La idea no sería automatizar todo, sino automatizar aquello que aporte valor: regresión, escenarios repetitivos, procesos críticos e integraciones donde sea importante detectar cambios rápidamente.

7. Performance

Para este ejercicio realizaría una prueba de carga sobre un servicio simulado localmente. El objetivo es comprobar que el servicio mantenga un comportamiento estable ante un volumen definido de solicitudes.

Criterios utilizados para la prueba:

- **Errores HTTP: menor al 1%.** El porcentaje de solicitudes que terminan con error HTTP debe ser inferior al 1%. Por ejemplo, sobre 1.000 solicitudes, 10 errores representan exactamente 1%, por lo que no cumplirían este criterio.
- **Errores funcionales: menor al 1%.** No basta con recibir HTTP 200; también se revisa que la respuesta tenga el contenido esperado, por ejemplo el estado de pago y el tracking ID.
- **P95 menor a 2 segundos.** El P95 indica el tiempo por debajo del cual se encuentra aproximadamente el 95% de las respuestas. En este caso, se busca que la gran mayoría responda dentro de los 2 segundos.

Estos valores son referencias iniciales para el ejercicio. En un proyecto real los validaría con el equipo, los SLA acordados y el volumen esperado del sistema.

Escenario de prueba:

- Servicio de pago simulado localmente.
- 10 solicitudes por segundo durante 10 segundos.
- Validación automática de errores y tiempos de respuesta.
- El pipeline debe fallar si no se cumplen los umbrales definidos.

8. CI/CD

La pipeline ejecutaría controles en diferentes etapas. La idea es detectar problemas lo antes posible y evitar que una versión avance si falla una validación importante.

- Instalación de dependencias.
- Validación de TypeScript.
- Auditoría de dependencias.
- Pruebas API.
- Pruebas transaccionales y asíncronas.
- Prueba E2E.
- Prueba de performance.
- Generación de evidencias cuando una ejecución termina.

9. Evidencia y criterios para liberar

No tomaría la decisión de liberar únicamente porque las pruebas pasaron. Revisaría los resultados, los defectos abiertos, los riesgos conocidos y si los escenarios críticos están cubiertos.

- Resultados de pruebas automatizadas.
- Defectos encontrados y su severidad.
- Cobertura de los escenarios críticos.
- Resultados de performance.
- Evidencias de integración y procesamiento asíncrono.
- Riesgos conocidos que deberían quedar documentados.

10. Matriz de trazabilidad

La trazabilidad permite relacionar un requisito o riesgo con la prueba que se realiza y con la evidencia obtenida. Esto ayuda a saber qué está cubierto y qué todavía queda pendiente.

Por ejemplo, si existe el riesgo de que una notificación de pago se procese dos veces, la matriz permite identificar la prueba que valida ese riesgo y dónde está la evidencia del resultado.

En el repositorio se relacionan elementos como:

- **Pago duplicado** > prueba transaccional de idempotencia > reporte de Playwright.
- **Mensaje duplicado** > prueba asíncrona > reporte de Playwright.
- **Comportamiento bajo carga** > prueba k6 > resultado y reporte de performance.
- **Acceso al portal** > prueba E2E > reporte de Playwright.

11. Métricas para los primeros 6 meses

Durante los primeros meses priorizaría métricas sencillas que permitan saber si la estrategia está mejorando la calidad y no solamente cuántas pruebas existen.

- Porcentaje de pruebas automatizadas sobre los escenarios críticos.
- Porcentaje de pruebas que pasan y fallan en cada ejecución.
- Defectos encontrados antes de producción.
- Defectos encontrados después de producción.
- Tiempo de ejecución de la regresión automatizada.
- Tasa de fallos repetitivos o pruebas inestables.

12. Seguridad y observabilidad

En las pruebas consideraría validaciones básicas para evitar exposición de datos sensibles, manejo incorrecto de credenciales y registros que permitan identificar información privada. Para diagnosticar fallos utilizaría el tracking ID y logs técnicos sin incluir datos sensibles.

13. Criterio general

La estrategia busca empezar por los riesgos de mayor impacto y avanzar gradualmente hacia una mayor automatización. La cantidad de pruebas no sería el único objetivo: también importa que sean confiables, mantenibles y que ayuden a tomar decisiones sobre la calidad del producto.